



5G NB-IoT 物联网综合应用实训指导书

V1.2.0

KE1-5G物联网实训

目 录

引言	4
文档说明.....	4
1. 术语表.....	4
2. 参考资料.....	4
3. 开发工具.....	4
NB-IoT 物联网综合应用实训指导	5
1. 实验目的.....	5
2. 实验环境.....	5
2.1 JAVA 运行环境安装.....	6
2.2 终端应用开发环境安装.....	6
2.3 移动应用 APP 开发环境安装	6
3. 实验原理.....	7
4. 实验内容.....	8
4.1 准备工作.....	8
4.2 AT 命令使用	8
4.2.1 AT 命令操作环境设置	9
4.2.2 NB 模组联网 AT 命令操作	11
4.3 移动端应用 APP 开发	13
4.3.1 终端平台注册演示界面(devreg.vue).....	14
4.3.2 终端控制（命令下发）演示界面(devctrl.vue).....	16
4.3.3 终端信息查看演示界面(devinfo.vue)	18
4.4 终端应用开发.....	20
4.4.1 KE1 开发板配置	21
4.4.2 终端应用开发说明	21
4.4.3 终端应用工程配置介绍	23
4.4.4 终端应用例程主要功能介绍	27
4.4.5 终端应用编译运行	34
4.5 KE1 微信小程序使用介绍	35
4.5.1 准备工作.....	35
4.5.2 KE1 小程序获取	35
4.5.3 KE1 小程序使用	36

	4.5.4 KE1 小程序功能介绍	36
5.	实验小结.....	39
6.	课后练习.....	39

KE1-5G物联网实训

引言

文档说明

- 1. 本文档为小熊座 KE1 5G 物联网(NB-IoT)综合应用实训指导手册
- 2. 版权所有，翻版必究

1. 术语表

定义本文的重要术语，为读者在阅读文档时提供必要的参考信息。

序号	术语或缩略语	说明性定义
1	NB-IoT	窄带物联网
2	STM	意法半导体公司芯片系列
3	APP	手机应用程序
4	KE1	小熊座 KE1 NB-IoT 物联网开发套装

2. 参考资料

资料名称	作者	用途
Quectel_BC35-G&BC28&BC95 R2.0_AT_Commands_Manual_V1.3	上海移远通信官方文档	AT 命令参考

3. 开发工具

- 开发语言：C、Vue.js、HTML、CSS
- 开发平台：Windows
- 应用开发工具：HBuilder X
- 终端嵌入式开发工具：STM32CubeIDE
- KE1 资料包下载地址：<https://www.lightimagesolutions.com/>

NB-IoT 物联网综合应用实训指导

1. 实验目的

本次实训，将通过一个完整的 NB-IoT 物联网综合实例，全方位解读物联网开发过程中所需要了解的基本知识点，并在实际操作过程中巩固并灵活运用已学的知识。

本实训需要掌握的知识点：

- 了解电信物联网云平台基本知识
- 掌握 NB 通信模组 AT 命令的使用
- 掌握嵌入式终端应用开发流程，并能开发自己的应用
- 掌握移动 APP 应用开发流程，并能开发自己的应用
- 熟悉物联网各个环节之间的关联和作用

2. 实验环境

本次实训过程中将会学习终端及移动应用端的软件开发，所以需要安装终端嵌入式开发环境和应用端开发环境，所需工具如下：

- ✓ 推荐自备 Windows 电脑
- ✓ 从 KE1 开发资源包中的[官方开发工具下载地址.txt](#)文件中获取相应的官方开发工具下载地址进行下载，并按照[帮助文档](#)目录中对应的工具安装帮助文档进行安装测试
- ✓ 需要下载的开发工具有：JAVA JRE、HBuilderX、STM32CubeIDE(版本 1.1.0)

2.1 JAVA 运行环境安装

1. 下载最新的 JRE 安装文件
2. 双击安装文件按照提示安装即可

2.2 终端应用开发环境安装

终端嵌入式开发环境安装步骤如下：

1. 解压所下载的 en.st-stm32cubeide_1.1.0.zip 文件到一个没有中文路径的目录。
(说明:终端例程使用的开发工具版本为1.1.0, 为了更好的兼容, 建议下载该版本)
2. 按照帮助文档“STM32 开发环境STM32CubeIDE 安装介绍.pdf”进行安装测试。

2.3 移动应用 APP 开发环境安装

移动应用 APP 开发环境安装步骤如下：

1. 解压所下载的最新 HBuilderX.x.x.x.xxxxx.full.zip 文件到任意磁盘根目录
2. 按照帮助文档“移动应用APP 开发环境HBuilderX 安装.pdf”进行安装测试

3. 实验原理

在终端，终端应用程序将读取 KE1 开发板上的温湿度传感器、声音传感器、光照强度传感器、电池电压等数值，并定时使用 AT 命令控制 NB 通信模组，通过 NB-IoT 网络向云平台南向接口上送信息。

在用户端，用户使用手机应用或者微信小程序等 KE1 应用 APP 从 KE1 云服务器中请求获取终端设备上送的各类传感器信息，应用 APP 对数据进行处理并展示在应用界面。并且用户也可以使用应用 APP 通过 KE1 云服务下发命令数据到终端，实现远程控制终端的目的。

通过移动端和终端应用的配合使用，最终实现物联网中最基本的数据采集，控制等操作。

KE1 5G 物联网综合实训网络架构图，如图 1

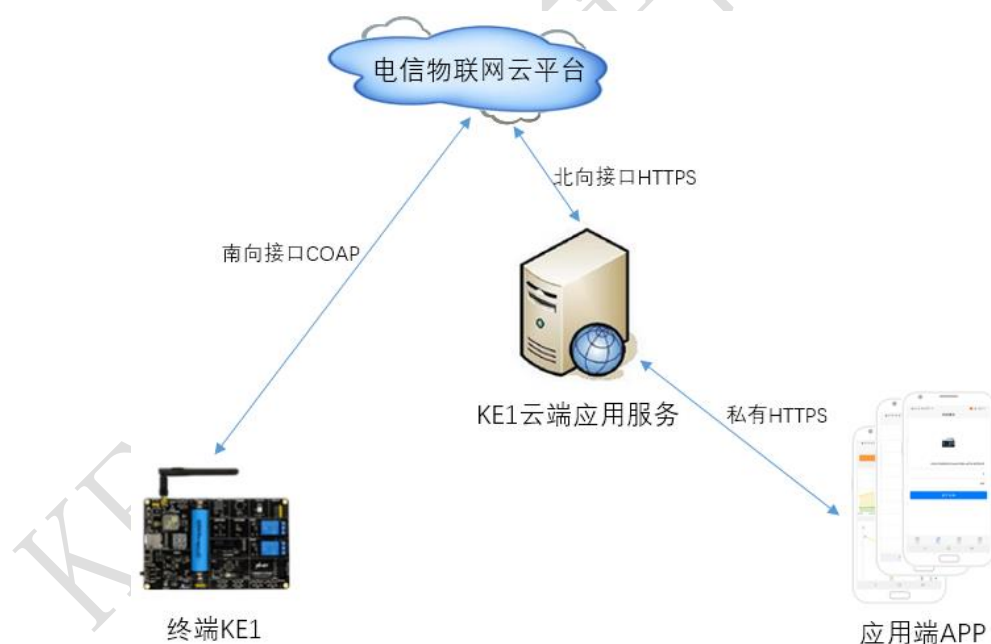


图 1 网络架构图

4. 实验内容

4.1 准备工作

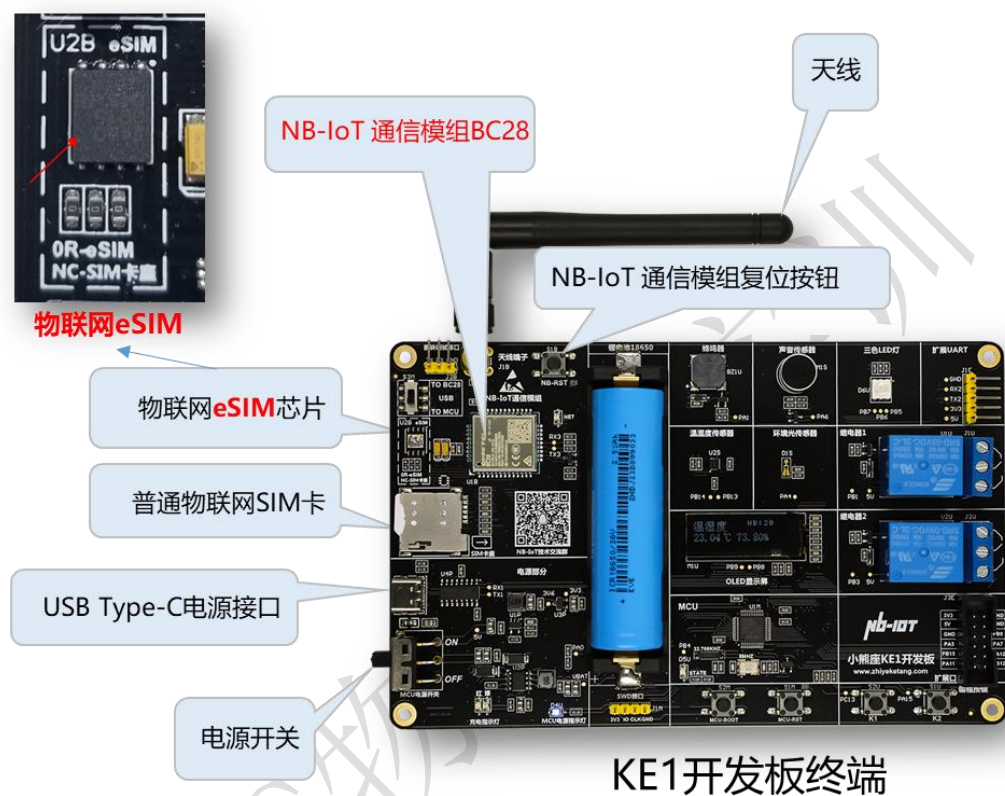


图 2 KE1 接口介绍

1. 检查 KE1 开发板是否已经安装 NB 物联网 eSIM 或者普通 NB 物联网 SIM 卡。(两者只需安装一个即可)
2. 检查是否已经安装天线。
3. 使用 USB Type-C 串口线给开发板供电。
4. 拨动电源开关到 NO 位置给开发板上电。

4.2 AT 命令使用

使用 AT 命令直接操作 NB 通信模组时，需要切换 KE1 开发板工作模式，并且需要使用 AT 串口调试助手 QCOM_V1.6。（在 KE1 开发资源包测试工具目录中获取）

4.2.1 AT 命令操作环境设置

1. 切换 KE1 开发板工作模式为 AT 命令测试模式。



图 3 切换工作模式到 TO-BC28

按照如图 3，将拨码开关切换到 **TO-BC28** 所在位置，并连接 KE1 开发板 USB 串口线到电脑端，此时 USB 串口线将直接连接到 KE1 开发板上的 NB 通信模组。

2. 检查电脑”设备管理器” 是否出现端口(COM)，如图 4

使用 USB 串口数据线将 KE1 开发板与电脑连接，并使 KE1 开发板上电。

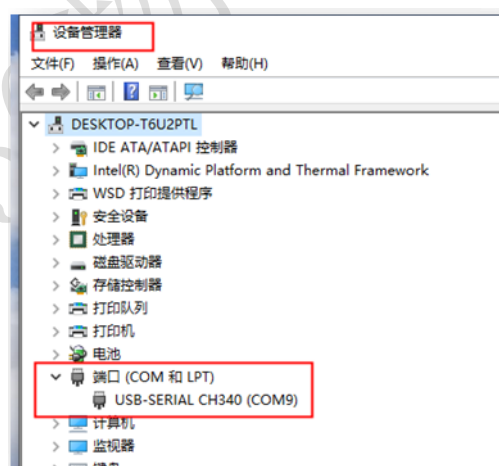


图 4 设备管理器串口号

3. 设置 AT 串口调试助手 QCOM_V1.6 工作参数

打开并配置 AT 串口调试助手参数(QCOM_V1.6.exe)。如图 5



图 5 AT 串口调试助手参数配置

- COM Port : 来自设备管理器(不同电脑的 COM 号可能不同)
 - Baudrate(波特率)设置为: 9600
 - 勾选” Send With Enter”
 - 其他参数默认即可
4. 点击 Open Port 启动串口调试助手。
 5. 在命令输入框输入 AT, 并点击 “Send Command” 按钮发送命令, 查看命令响应框中是否能收到 OK 回复, 如图 5, 如果有说明连接正常, 否则请重新检查配置。

4.2.2 NB 模组联网 AT 命令操作

演示如何通过 AT 命令的方式配置 NB 通信模组联网到电信物联网云平台。依次按照如下步骤发送 AT 命令到 NB 通信模组，如图 6。

1. 发送命令 ATE1 启动通信模组串口回显功能
2. 发送命令 AT+CFUN=0 关闭通信模组
3. 发送命令 AT+NCDP=180.101.147.115,5683 设置电信平台地址
4. 发送命令 AT+CFUN=1 启动通信模组
5. 发送命令 AT+CGDCONT=1,"IP","CTNB" 设置 PDP
6. 发送命令 AT+CGATT=1 启动通信模组联网
7. 发送命令 AT+CGATT? 查看联网状态是否返回 1，返回 1 代表联网成功，否则表示连接中，连接成功后即可操作接下来的 AT 命令
8. 发送命令 AT+CGSN=1 获取模组 IMEI 号（可用于终端在电信物联网云平台的注册，参考 [4.1.3](#)）
9. 发送命令 AT+CSQ 查看信号值
10. 发送命令 AT+CEDRXS=0,5 关闭模组 eDRX 功能(退出低功耗模式)
11. 发送命令 AT+CPSMS=0 关闭模组 PSM 功能(退出低功耗模式)
12. 模拟上送设备温度(T)湿度(H)信息{"T":"12","H":"96"}，该字符串对应的 ASCII 码为：(HEX 格式) **7B2254223A223132222C2248223A223936227D**，

字符串长度为 19，对应的十六进制为 0x**0013**，数据上送的 AT 命令如下：

AT+NMGS=22,000013**7B2254223A223132222C2248223A223936227D**

本例数据格式为：messageId(00)+信息字符长度(**0013**)+信息字符串

（该命令需要使用例程应用 APP 或者微信小程序注册绑定设备后才可使用，请参考章节 [4.1.3](#) 或者 [4.1.5](#)）

AT 命令参考文档

Quectel_BC35-G&BC28&BC95 R2.0_AT_Commands_Manual_V1.3.pdf

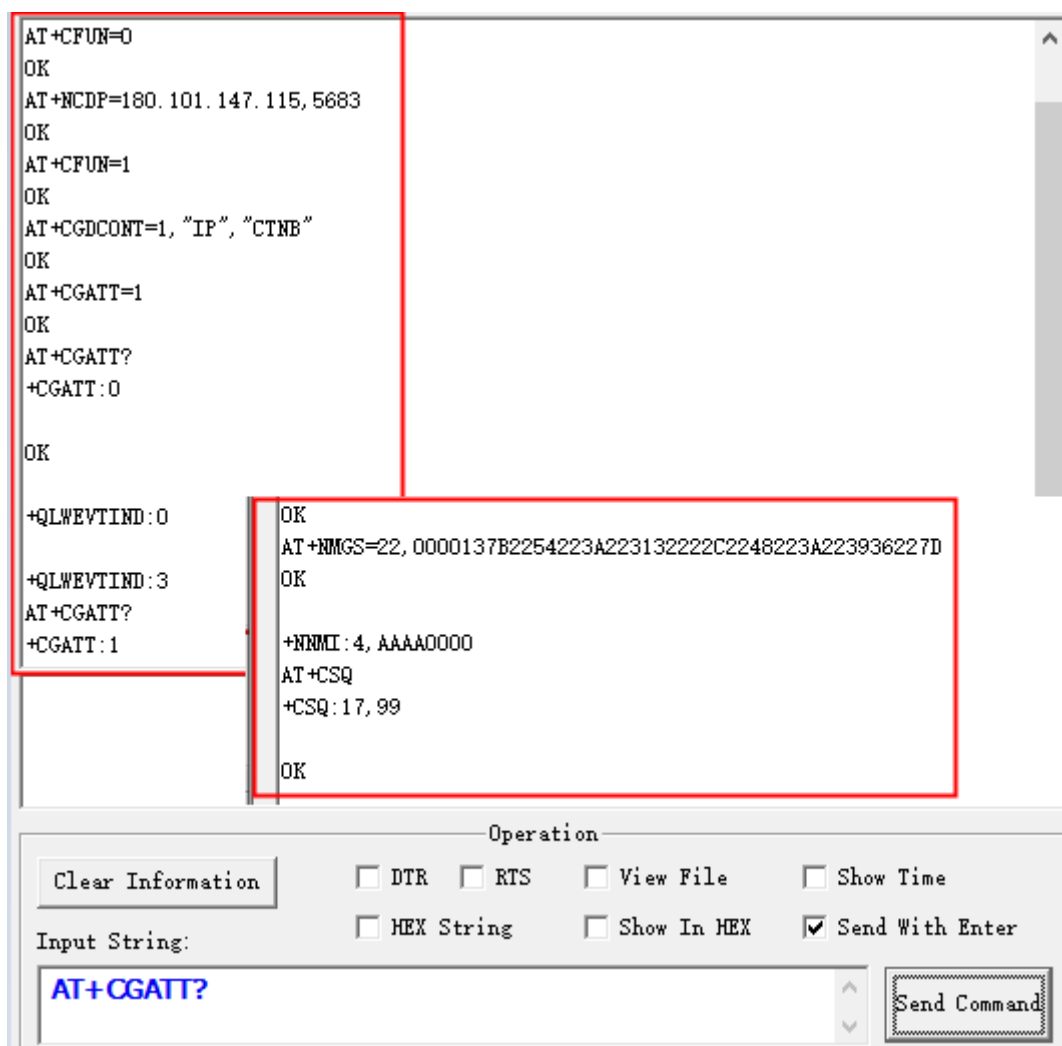


图 6 NB 模组 AT 命令联网

使用 AT 命令时可能遇到的错误以及解决办法:

1. AT 串口调试助手接收到乱码

请查看波特率是否设置为 9600。

2. NB 模组始终无法联网，或者信号值为 99

请查看是否安装天线以及物联网 SIM 卡。

3. 发送的 AT 命令出现 ERROR

可重启 NB 模组（按一下模组旁边的 NB-RST 键）并按照规定发送 AT 命令。另外有些 AT 命令需要使用 AT+CFUN=1 启动模组后才可以。

4. 使用 AT+NMGS 上送信息时出现 ERROR

可能上送信息频率过快（建议每隔 15 秒上送一次信息），或者重启 NB 模组进行网络连接。

4.3 移动端应用 APP 开发

通过章节 [2.3](#) 成功安装移动应用开发环境后, 启动开发工具 HBuilder X 并导入例程源码 KE1APP。如图 7 (在 KE1 开发资源包例程源码目录中获取)

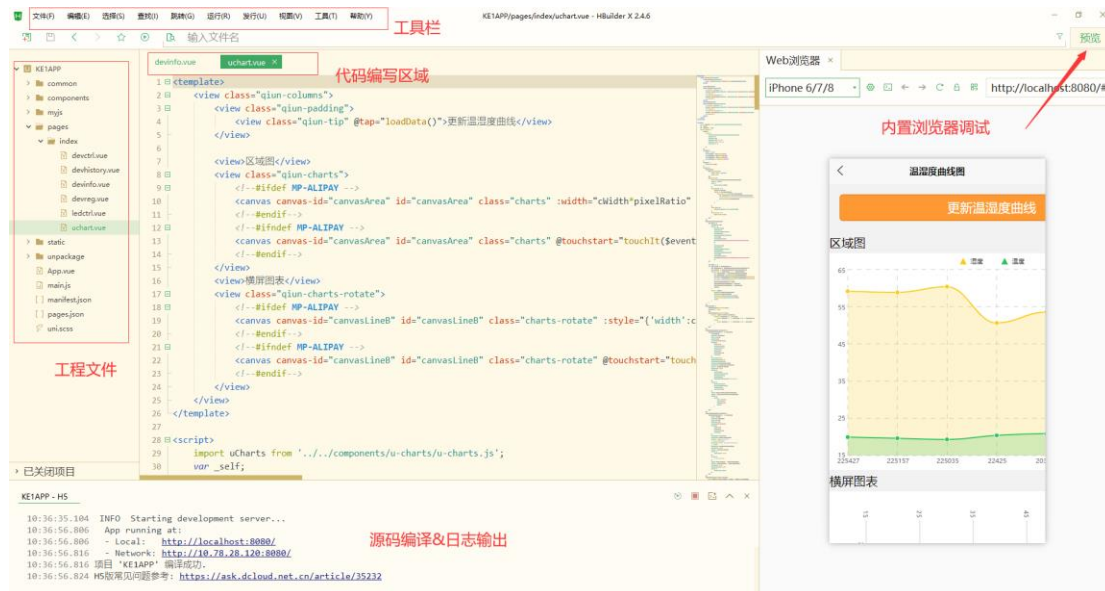


图 7 HBuilder 开发工具界面

通过工具简介, 我们可以尝试预览并熟悉例程应用 APP 各个界面功能。该例程有三个主要功能界面

1. 终端平台注册演示界面(devreg.vue)
2. 终端控制 (命令下发) 演示界面(devctrl.vue)
3. 终端信息查看演示界面(devinfo.vue)

可直接使用安卓手机浏览器扫描下方二维码下载 APP



4.3.1 终端平台注册演示界面(devreg.vue)



图 8 终端平台注册演示界面

该界面主要功能是对终端设备进行电信物联网云平台注册。注册方法如下：

1. 手动输入 KE1 开发板 NB 模组的 IMEI 号（通过 AT 命令 **AT+CGSN=1** 获取），或者编译安装 APP 到手机，点击界面扫码图标，APP 将调用手机摄像头扫描模组上方的二维码自动获取 IMEI 号。如图 9



图 9 模组 IMEI 号二维码位置

2. 点击界面“注册添加设备”按钮进行电信物联网云平台注册

注册成功后，电信物联网云平台将会返回一个唯一的设备 ID 号，该设备 ID 号需要记录并替换其它所有.vue 界面源码中的默认 ID 值，如图 10



图 10 需要替换设备 ID 的文件和位置

3. 使用 AT 命令上送设备端信息，参考章节 [4.2.2](#)

使用该界面注册设备时需要注意以下事项：

1. 不能随意输入 IMEI 号进行注册，避免无法获取正确的设备 ID。
2. 设备注册成功后，只有当 KE1 终端使用 AT 命令上送了设备信息后才可以应用 APP 上查看设备信息。

4.3.2 终端控制（命令下发）演示界面(devctrl.vue)

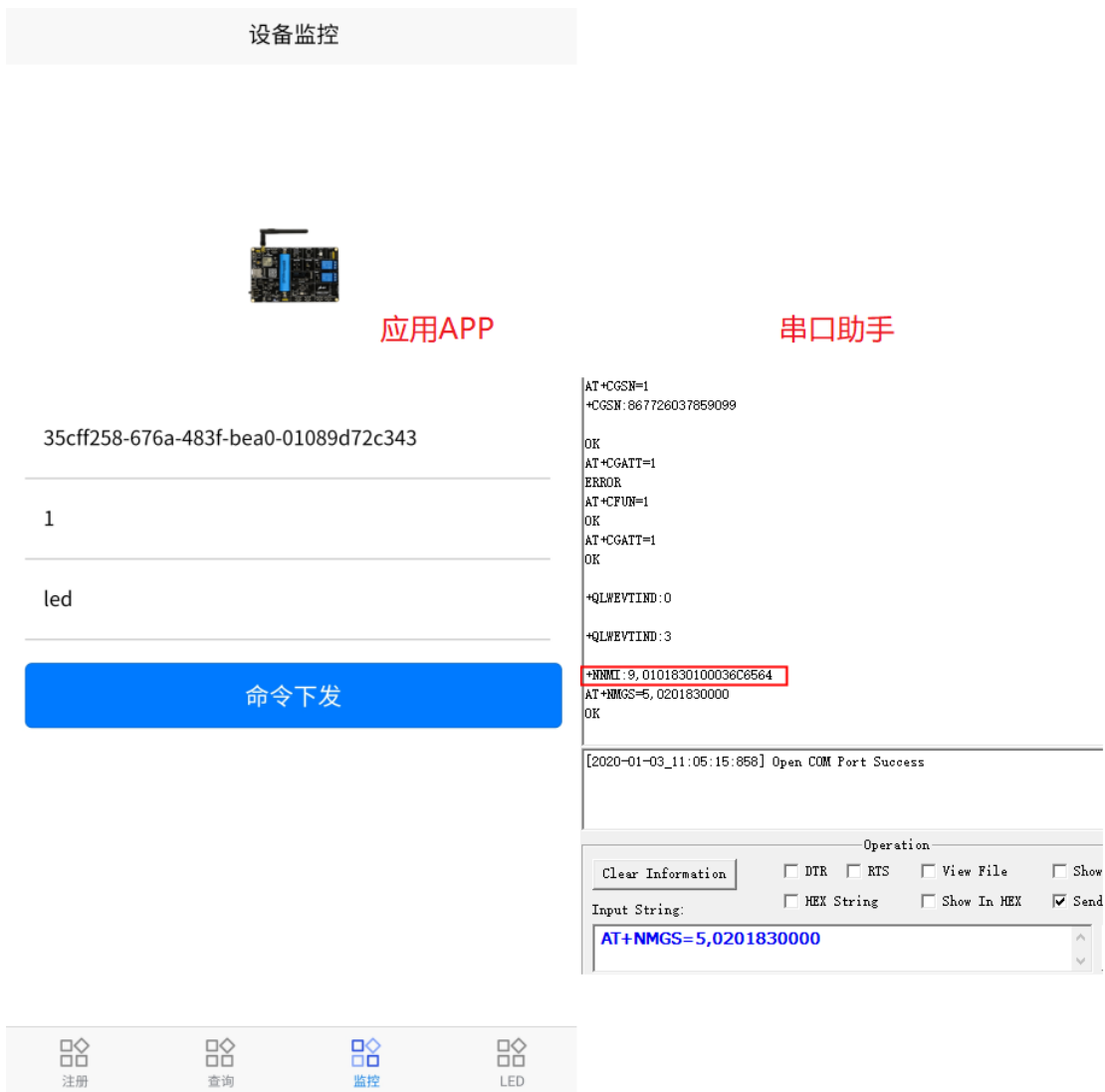


图 11 终端控制界面

该界面主要功能是演示通过电信物联网云平台下发命令数据，从而实现对设备端的远程控制。使用方法如下：

输入正确的设备 ID（注册设备时所获取的），以及所需要下发的命令码和命令字符串，点击“命令下发”按钮，应用 APP 将请求 KE1 云端服务通过电信物联网平台下发用户输入的命令码和命令字符串，此时我们可以通过 AT 串口调试助手查看终端程序是否收到命令数据。例如命令码为 1，命令字符串为“led”，终端模组接收到信息后将自动返回如下数据：

+NNMI:9,0101830100036C6564 下发命令数据格式：messageId(01)+MID(0183)+命令码(01)+命令字符串长度(0003 (HEX))+命令字符串（led 的 ASCII 码 6C6564 (HEX)）

终端在接收到下发命令后，需要发送 AT 命令，来响应该命令执行结果（成功或者失败等信息），例如响应上述命令的 AT 命令如：AT+NMGS=5,0201830000

命令响应数据由 messageId(02)+MID(0183,该值需要与下发命令的 MID 一致)+响应码(00)+执行状态码(00)

当前，KE1 云端服务在电信物联网云平台配置的命令服务属性支持命令码（整型 int）和命令字符串（string）类型的数据。其中，命令码支持 0~255 的数字，命令字符串支持用户自定义任意命令内容（不支持中文，命令字符串长度限制在 128 字节）。

使用该界面下发控制命令时需要注意以下事项：

1. 命令下发不能太频繁，本例应用 APP 限制 1 分钟下发一次，以避免网络堵塞。
2. 如果终端 NB 通信模组始终无法接收到命令数据，可能 NB 通信模组已进入睡眠模式，可以尝试重启终端应用程序或者重启模组。

4.3.3 终端信息查看演示界面(devinfo.vue)



图 12 终端信息查询

该界面主要功能是演示通过电信物联网云平台获取终端设备上送的信息，并通过三种形式在 APP 界面进行展示，方便用户查看分析数据。

其中，点击“信息查询”按钮，应用 APP 将请求 KE1 云端服务从电信物联网云平台获取设备最新一次上送的信息，解析并显时出终端所上送信息的温湿度值，同时显示出电信物联网云平台返回的原始 JSON 数据，可用于参考学习。如图 12

点击“历史信息查询”按钮，应用 APP 将请求 KE1 云端服务从电信物联网云平台获取设备最近 30 条上送信息的温湿度历史记录（KE1 云端服务限制最多返回 30 条历史记录），并以列表的新式显示在应用 APP 上。如图 13

设备历史信息
温度: 12°C 湿度: 96%
时间: 20200102T055443Z
温度: 19.94°C 湿度: 59.08%
时间: 20191225T065427Z
温度: 19.60°C 湿度: 58.78%
时间: 20191225T065157Z
温度: 19.38°C 湿度: 60.36%
时间: 20191225T065035Z
温度: 20.48°C 湿度: 50.57%
时间: 20191225T060425Z
温度: 20.88°C 湿度: 53.57%
时间: 20191225T043921Z
温度: 19.32°C 湿度: 46.59%
时间: 20191220T102922Z
温度: 18.76°C 湿度: 45.75%
时间: 20191220T082925Z
温度: 17.94°C 湿度: 47.35%
时间: 20191220T062928Z
温度: 16.30°C 湿度: 47.01%
时间: 20191220T042930Z

图 13 设备历史信息列表

点击“图表查看”按钮，应用 APP 将以图表形式展示温湿度曲线图，如图 14

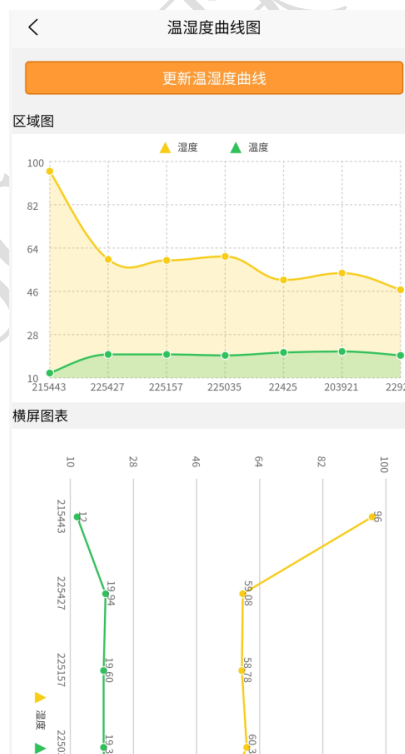


图 14 设备历史信息图表

本实例中，终端上送的字符串数据为 JSON 数据格式。例如 “{“T”:“12”,“H”:“96”}”。

终端上送的信息里可以继续添加符合 JSON 格式的其它信息，例如增加噪音值 “{“T”:“12”,“H”:“96”,“S”:“236”}”，此时应用 APP 需要做相应的数据解析并显示。

4.4 终端应用开发

在应用 APP 开发章节中，应用 APP 获取的数据都是以 AT 串口工具手动发送 AT 命令的形式进行终端信息上送操作，这在实际应用场景中是非常不可取的。

在真实的物联网应用场景中，终端 MCU 将运行终端应用程序，通过编程读取相关传感器等外设的信息，并使用 MCU 串口发送 AT 命令控制通信模组自动上送信息，并处理其它终端应用逻辑。

本实例将全面了解终端应用开发的过程，熟悉终端嵌入式开发工具的使用，熟悉如何使用 MCU 串口发送 AT 命令控制通信模组等等...

在章节 [2.2](#) 中，成功安装嵌入式开发环境 STM32CubeIDE，启动该工具并导入例程源码 KEISouth(在 KE1 开发资源包例程源码目录中获取)。例程工作主流程图。如图 15

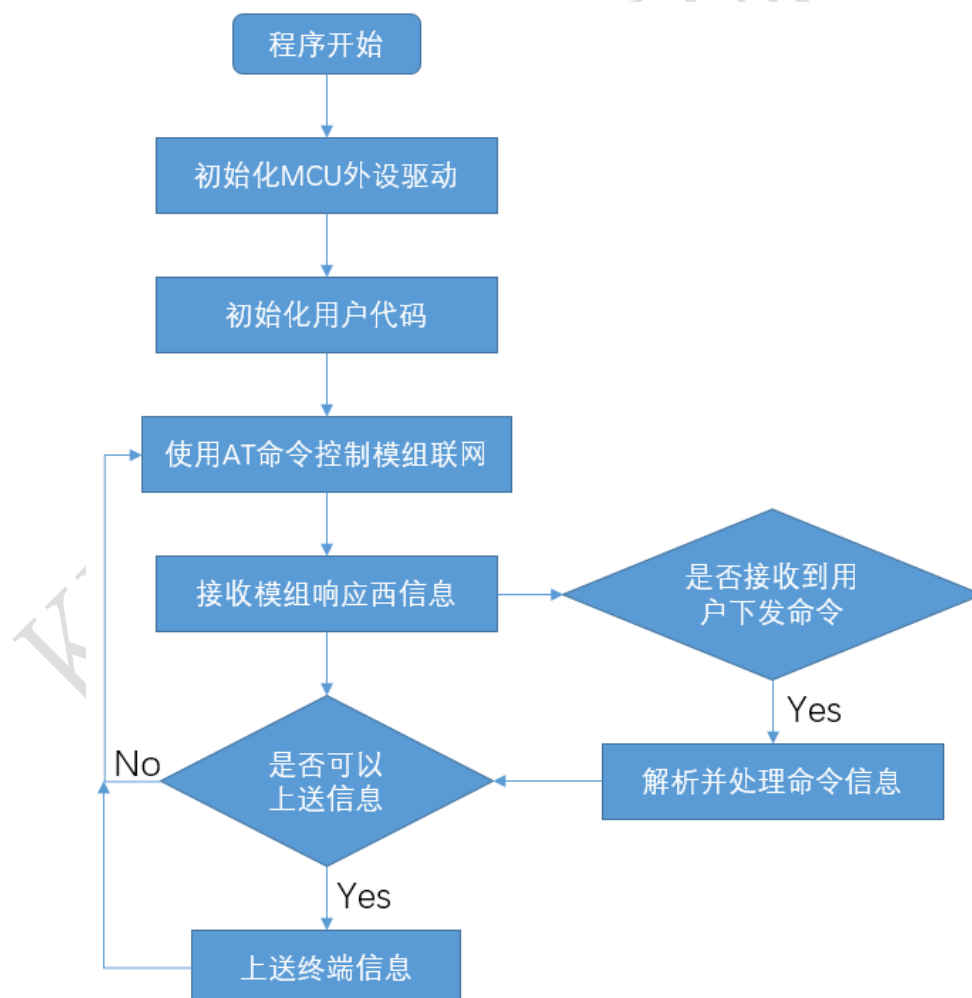


图 15 终端程序主流程图

4.4.1 KE1 开发板配置

按照如图 16 方式，确保将拨码开关切换到 **TO-MCU** 所在位置，此时 MCU 的串口（USART3）将连接到通信模组，用于 MCU 应用程序进行 AT 命令控制。连接 KE1 开发板 USB 串口线到电脑端，设置串口工具的波特率为 **115200**（MCU 调试串口 USART1 的波特率），上电 KE1 开发板。



图 16 切换 KE1 开发板到 MCU 工作模式

4.4.2 终端应用开发说明

终端例程开发环境为 STM32CubeIDE，KE1 开发板所使用的 MCU 芯片型号为 STM32L433CBT6，使用的是 STM32 HAL 库函数。编写程序时需要遵循如下规则：

1. 在 STM32CubeIDE 自动生成的源码文件中，如果需要添加自己的代码，那么必须编写在该文件中的 `/*USER CODE BEGIN xxxx */` 和 `/* USER CODE END xxx */` 之间。如图 17
2. STM32CubeIDE 自带参考例程源码，通过以下路径获取本机例程源码进行学习。

C:\Users\your_pc_name\STM32Cube\Repository\STM32Cube_FW_L4_V1.14.0\Projects\N
UCLEO-L432KC\Examples

```
139
140 /* USER CODE END SysInit */
141
142 /* Initialize all configured peripherals */
143 MX_GPIO_Init();
144 MX_DMA_Init();
145 MX_ADC1_Init();
146 MX_USART1_UART_Init();
147 MX_USART3_UART_Init();
148 MX_I2C2_Init();
149 MX_TIM2_Init();
150 /* USER CODE BEGIN */
151 Beep_Switch(1);
152 HAL_Delay(500);
153 Beep_Switch(0);
154
155 UART_Enable_Receive_IT(); // 使能串口接收
156
157 OLED_Init(); // 初始化OLDE
158
159 OLED_DrawLogo(); // 显示bmp单色logo图片
160 HAL_Delay(2000);
161 OLED_ShowString(0, 0, (uint8_t *) "OLED");
162 HAL_Delay(2000);
163
164 OLED_ShowKE1(); // 显示小熊图KE1
165 HAL_Delay(2000);
166
167 OLED_Clear(); // 清屏
168
169 printf("Hello! i am KE1 south demo V0.0.1\n");
170 HAL_Delay(500);
171 KE1_I2C_SHT31(&fTemp, &fHumi);
172 printf("T:%0.2f, H:%0.2f%\r\n", fTemp, fHumi);
173 KE1_ADC_Sensor_Get(&usLight, &usSound);
174 printf("L:%d, S:%d%\r\n", usLight, usSound);
175 OLED_ShowT_H(fTemp, fHumi); // OLED显示温度湿度
176
177 /* USER CODE END */
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

图 17 用户代码编写区域

4.4.3 终端应用工程配置介绍

- MCU 管脚基础配置说明，如图 18
 1. 三色 LED 灯：PB4, PB5, PB6, PB7 配置为输出模式 GPIO_Output
 2. OLED 驱动：PB8, PB9 配置为输出模式 GPIO_Output，用于 I2C 协议模拟
 3. 温湿度驱动：PB13 配置为 I2C2_SCL，PB14 配置为 I2C2_SDA
 4. 按键 K1：PC13 配置为 GPIO_EXTI13，外部中断模式
 5. 按键 K2：PA15 配置为 GPIO_EXTI15，外部中断模式
 6. 声音传感器：PA6 配置为 ADC1_IN11，模数转换
 7. 环境光传感器：PA4 配置为 ADC1_IN9，模数转换
 8. 电池电压：PA0 配置为 ADC1_IN5，模数转换
 9. 通信模组控制串口(UART3):PB10 配置为 USART3_TX,PB11 配置为 USART3_RX
 10. MCU 程序调试串口(UART1):PA9 配置为 USART3_TX,PA10 配置为 USART3_RX
- MCU 系统核心配置说明
 1. 使能串口（USART1&USART3）中断和外部中断（EXTI line）。如图 19
 2. 添加 ADC1 支持 DMA 功能。如图 20
 3. GPIO 配置 PC13, PA15 为默认上拉模式（Pull-up）。如图 21

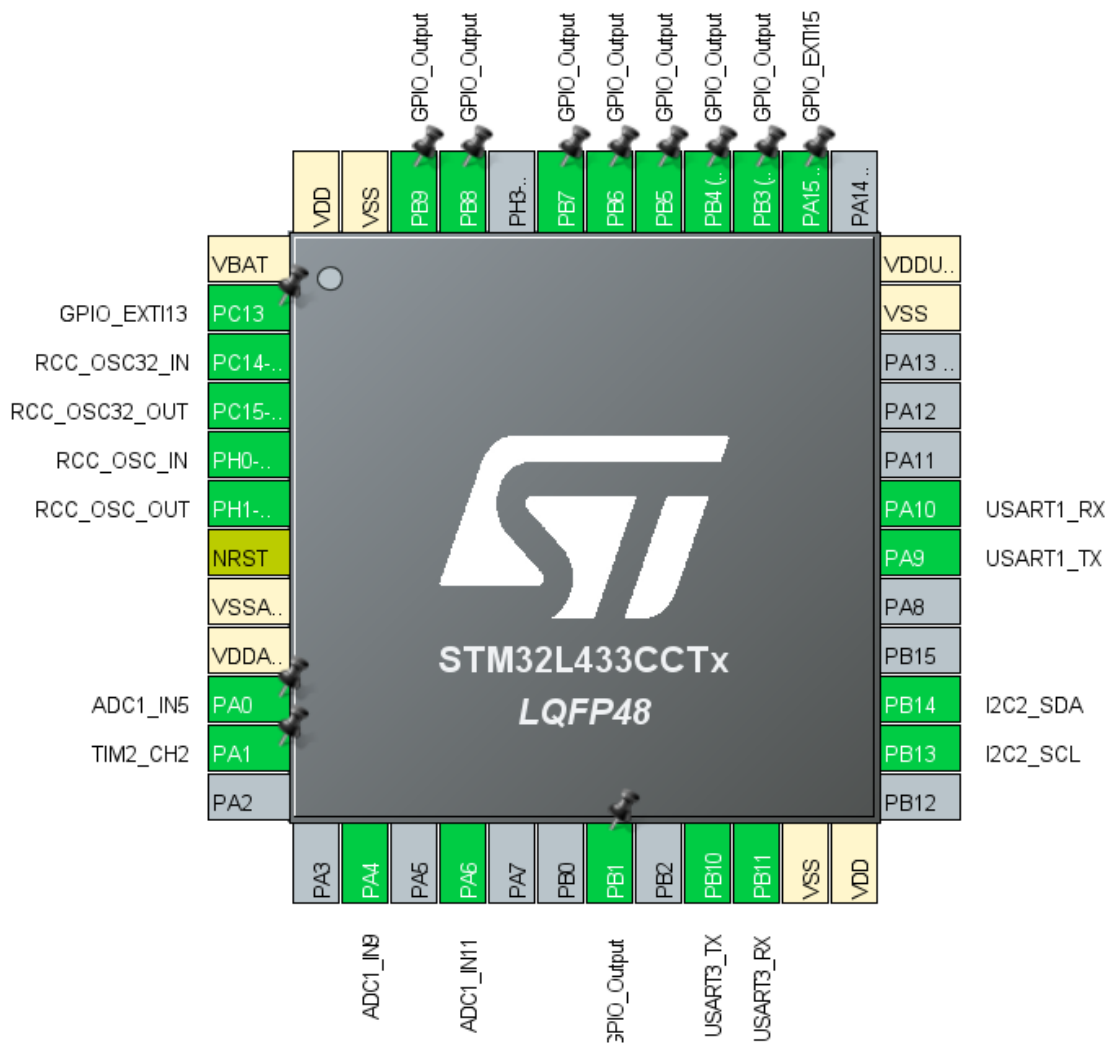


图 18 MCU 管脚基础配置

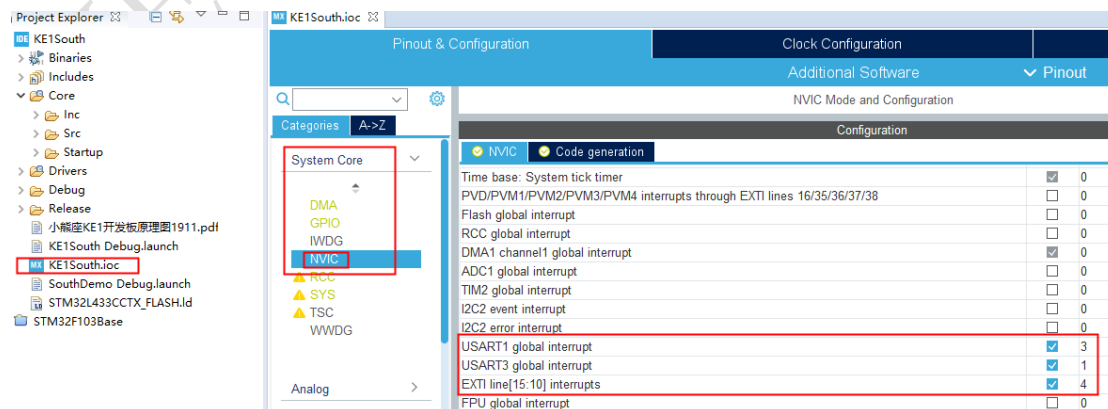


图 19 MCU 系统核心配置

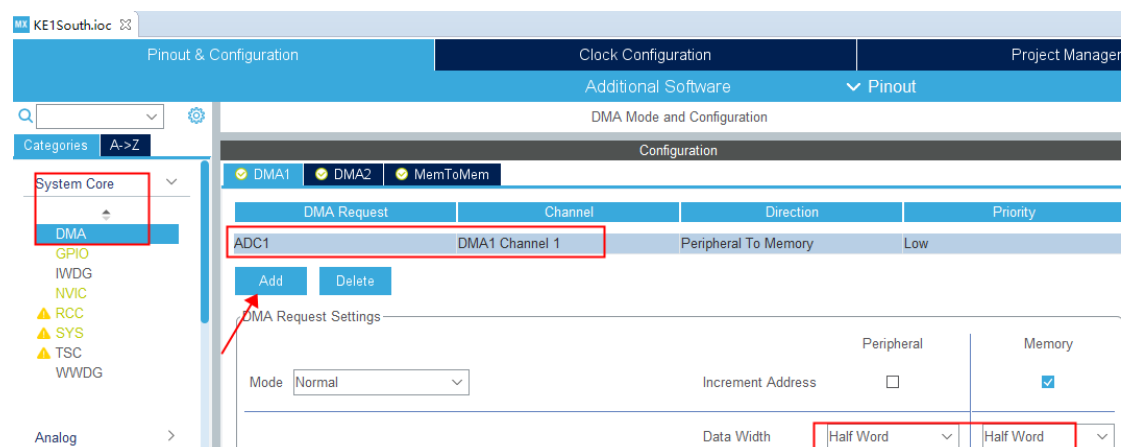


图 20 添加 DMA

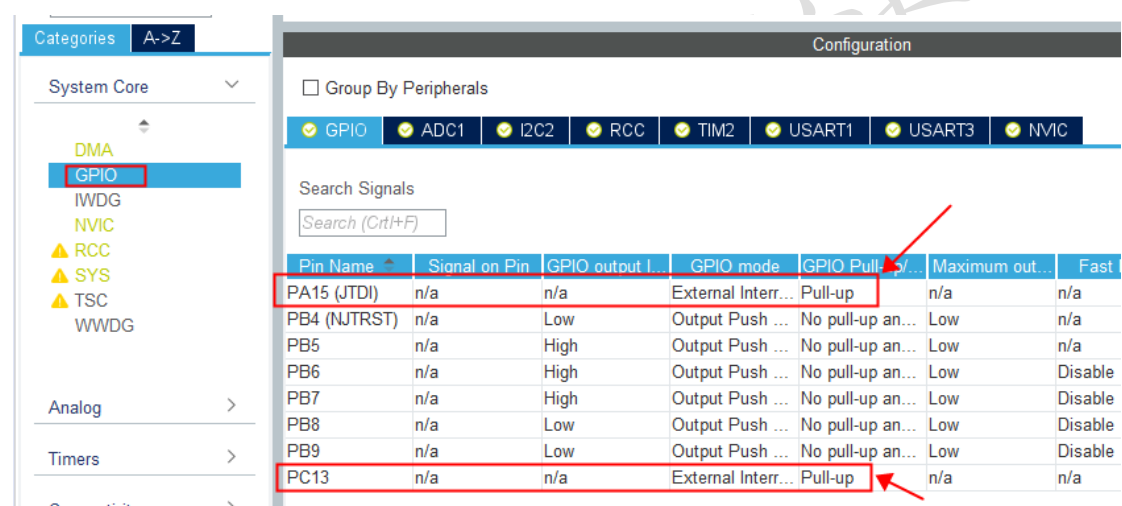


图 21 GPIO 配置

● MCU 通信接口配置，如图 22

1. 使能 I2C2，用于温湿度芯片。
2. 设置 USART1 通信参数，其中波特率为 115200，用于程序调试输出。
3. 设置 USART3 通信参数，其中波特率为 9600，用于程序对通信模组进行 AT 命令收发。

KE1South.ioc | main.c | usart.c

Pinout & Configuration | Clock Configuration | Additional Software

Categories | A-Z

Analog >

Timers >

Connectivity >

- CAN1
- I2C1
- I2C2
- I2C3
- IRTIM
- LPUART1
- QUADSPI
- SPI1
- SPI2
- SPI3
- SWPMI1
- USART1
- USART2
- USART3
- USB

USART1 Mode and Configuration

Mode

Mode: Asynchronous

Hardware Flow Control (RS232): Disable

☐ Hardware Flow Control (RS485)

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Baud Rate: 115200 Bits/s

Word Length: 8 Bits (including Parity)

Parity: None

Stop Bits: 1

Advanced Parameters

Data Direction: Receive and Transmit

Over Sampling: 16 Samples

Basic Parameters

Baud Rate: 9600 Bits/s

Word Length: 8 Bits (including Parity)

Parity: None

Stop Bits: 1

Advanced Parameters

Data Direction: Receive and Transmit

Over Sampling: 16 Samples

图 22 MCU 通信接口配置

4.4.4 终端应用例程主要功能介绍

- 温湿度获取函数

```
/*
 * 函数说明: 获取温湿度
 * 输入参数
 * pfTemp: 温度输出地址
 * pfHumi: 湿度输出地址
 * 返回值
 * 0: 成功
 */
int KE1_I2C_SHT31(float *pfTemp, float *pfHumi)
{
    unsigned short usVal = 0;
    unsigned char dataBuf[6] = {0};
    if(NULL == pfTemp || NULL == pfHumi) return -1;
    *pfTemp = 0.0;
    *pfHumi = 0.0;
    /*
    参数说明
    0x89 -> I2C 芯片总线地址
    0x240B -> I2C 芯片命令(读取数据)
    */
    HAL_I2C_Mem_Read(&hi2c2, 0x89, 0x240B, I2C_MEMADD_SIZE_16BIT, dataBuf, 6,
0xFFFF);

    usVal = (dataBuf[0]<<8)|dataBuf[1];
    *pfTemp = 175.0f*(float)usVal/65535.0f-45.0f;

    usVal = (dataBuf[3]<<8)|dataBuf[4];
    *pfHumi = 100.0f*(float)usVal/65535.0f;

    return 0;
}
```

函数实现在工程 i2.c 源文件中。

KE1 开发板使用的是 SHT3x 温湿度传感器，该传感器湿度测量精度 $\pm 1.5\%RH$ ，温度测量精度在 $\pm 0.2\text{ }^{\circ}\text{C}$ ，具体信息可参考芯片资料 **C80862_SHT3X.pdf**。函数调用方法：

```
float fTemp = 34.2, fHumi = 0.0;
KE1_I2C_SHT31(&fTemp, &fHumi);
printf("T:%0.2f, H:%0.2f%%\r\n", fTemp, fHumi);
```

● 噪音和光照强度获取函数

```
/*
* 函数说明: 获取光照强度以及噪音值（所获值仅供参考）
* 输入参数
* pusLight: 光照强度输出地址
* pusSound: 噪音输出地址
* 返回值
* 0: 成功
*/
int KE1_ADC_Senser_Get(unsigned short *pusLight, unsigned short *pusSound, unsigned
short *pusVoltage)
{
    static unsigned short ausAdcDataBuf[2] = {0};
    unsigned short val = 0;
    float fVolts = 0.0, fAmps = 0.0, fMicroamps = 0.0, fLux = 0.0;

    *pusLight = 0;
    *pusSound = 0;

    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)ausAdcDataBuf, 2); // 开始 ADC DMA 采样
    HAL_Delay(100); // 延时 100ms
    HAL_ADC_Stop(&hadc1); // 结束 AD DMA 采样

    // 光照强度 AD 数据转换（仅供参考）
    fVolts = ((3.3f*ausAdcDataBuf[0])/4096.0f);
    fAmps = fVolts / 10000.0f;
    fMicroamps = fAmps * 1000000;
    fLux = fMicroamps * 2.0f;
    printf("Light:PA4_ADC:%d - %.2fLux\n", ausAdcDataBuf[0], fLux);
    *pusLight = (uint16_t)fLux;

    // 噪音 AD 数据转换（仅供参考）
    val = ausAdcDataBuf[1];
    val = (((3.3f*val)/4096.0f)*2*1.5f)*1000;
    printf("Sound:PA6_ADC:%d - %d\n", ausAdcDataBuf[1], val);
    *pusSound = val;

    // 电池 AD 数据转换（参考）
    val = ausAdcDataBuf[2];
    val = ((val * 825)>>10)*3;//(val *
3300)/4096;//(val*2.17f);//(((3.3f*val)/4096.0f)*2.5f)*1000;
    *pusVoltage = val;
    return 0;
}
```

函数实现在工程 adc.c 源文件中。函数调用方法：

```
unsigned short usLight = 0, usSound = 0 , usVoltage = 0;
KE1_ADC_Senser_Get(&usLight, &usSound, &usVoltage);
```

- MCU 串口 UART3 发送 AT 命令到通信模组

```
/*
 * 函数说明: 向模组发送指定 AT 命令
 * 输入参数
 * pcAt: AT 命令
 * 返回值
 * 无返回值
 */
void KE1_Send_AT(char *pcAt)
{
    HAL_StatusTypeDef sta;
    sta = HAL_UART_Transmit_IT(&huart3, (uint8_t *)pcAt, strlen(pcAt));
    if(HAL_OK != sta){
        printf("uart3 err:%d\n", sta);
    }
}
```

函数实现在工程 usart.c 源文件中。函数调用方法：

```
char *pcAt = "AT+NCDP=180.101.147.115,5683\r\n";
KE1_Send_AT(pcAt);
```

使用该函数时，所需发送的 AT 命令字符串后面必须带有 `\r\n` 回车换行符

- MCU 串口 UART3 接收并解析部分 AT 命令响应

```
/*
 * 函数说明: 接收并解析模组返回的 AT 响应数据
 * 输入参数
 * pcOut: 输入输出参数, 输入接收数据缓存地址, 输出已经接收的数据
 * puiSz: 输入输出参数, 输入接收缓存长度, 输出已经接收的数据长度
 * uiTimeoutMs: 输入参数, 接收超时(ms)
 * 返回值
 * -1 出错
 * 0 AT 命令响应超时
 * 1 AT 命令接收到 OK 响应
 * 2 AT 命令接收到 ERROR 响应
 * 3 接收到网络已注册提示
 * 4 模组接收到云端下发数据
 */
```

```

int KE1_Rcv_AT(char *pcOut, unsigned int *puiSz, unsigned int uiTimeoutMs)
{
    unsigned char OK[] = "OK\r\n", okT = 0, ERROR[] = "ERROR\r\n", errT = 0;
    unsigned char NNMI[] = "+NNMI:", nnmiT = 0;
    unsigned char CGATT[] = "+QLWEVTIND:3\r\n", cgattT = 0;
    int ch = -1, len = 0;
    unsigned int uiSz = *puiSz;
    *puiSz = 0;
    if(NULL == pcOut || NULL == puiSz) return -1;

    do{
        ch = KE1_Get_NB_Ch();
        if(-1 != ch){
            KE1_Put_Console_Ch(ch);
            if(len != uiSz){
                pcOut[len++] = ch;
                *puiSz = len;
                pcOut[len] = 0;
            }
            if(4 != okT){
                if(OK[okT] == ch){
                    okT++; if(4 == okT){return 1;}
                }else{
                    okT = 0;
                }
            }
            if(7 != errT){
                if(ERROR[errT] == ch){
                    errT++; if(7 == errT){return 2;}
                }else{
                    errT = 0;
                }
            }
            if(14 != cgattT){
                if(CGATT[cgattT] == ch){
                    cgattT++; if(14 == cgattT){return 3;}
                }else{
                    cgattT = 0;
                }
            }
            if(6 != nnmiT){
                if(NNMI[nnmiT] == ch){
                    nnmiT++;
                }else{

```

```

        nnmiT = 0;
    }
} else {
    if(0x0A == ch){
        return 4;
    }
}
} else {
    if(0 != uiTimeoutMs) {
        uiTimeoutMs--;
        HAL_Delay(1);
        continue;
    }
}
} while(uiTimeoutMs);
return 0;
}

```

函数实现在工程 usart.c 源文件中。函数调用方法：

```

int iRet = -1;
unsigned int atLen = 0 , timeout = 1000;
char acAtBuf[512] = {0};
atLen = sizeof(acAtBuf);
iRet = KE1_Rcv_AT(acAtBuf, &atLen, timeout);
if(0 == iRet){ //AT 命令响应超时
    printf("AT Timeout\r\n");
} else if(1 == iRet){ //AT 命令接收到 OK 响应
    printf("AT OK !\r\n");
} else if(2 == iRet){ //AT 命令接收到 ERROR 响应
    printf("AT ERROR !\r\n");
} else if(3 == iRet){ //接收到网络已注册提示
    printf("Net ready !\r\n");
} else if(4 == iRet){ //AT 命令接收到电信物联网平台下发数据
    printf("got user command !\r\n");
}
}

```

- 使用 AT 命令控制通信模组联网

```
switch(iUserCase){
    case 0:
        KE1_Clear_AT_Buf();
        break;
    case 1:
        KE1_Send_AT("AT\r\n"); // AT 测试模组
        break;
    case 2:
        KE1_Send_AT("AT+CFUN=0\r\n"); // 关闭模组
        break;
    case 3: // 设置电信物联网南向接口地址
        KE1_Send_AT("AT+NCDP=180.101.147.115,5683\r\n");
        break;
    case 4:
        KE1_Send_AT("AT+CFUN=1\r\n"); // 启动模块
        break;
    case 5:
        KE1_Send_AT("AT+CGDCONT=1,\"IP\",\"CTNB\"\r\n"); // 设置 PDP
        break;
    case 6:
        KE1_Send_AT("AT+CGATT=1\r\n"); // 模块开始注册网络
        break;
}
iUserCase++;
```

该代码演示了使用 MCU 自动发送 AT 命令进行通信模组联网的基本过程，具体逻辑可参考工程主函数 main.c 源文件。

- 使用 AT 命令上送信息

```
KE1_I2C_SHT31(&fTemp, &fHumi); // 采集温湿度或者其他信息上传到电信物联网平台
KE1_ADC_Senser_Get(&usLight, &usSound);
dLen = snprintf(acDevInfo, sizeof(acDevInfo),
    "{ \"T\": \"%0.2f\", \"H\": \"%0.2f\", \"L\": \"%d\", \"S\": \"%d\" }", fTemp, fHumi,
    usLight, usSound); // 打包用户数据
printf("%s\r\n", acDevInfo);
ascii2hex(acDevInfo, acHexBuf);
snprintf(acAtBuf, sizeof(acAtBuf), "AT+NMGS=%d,00%04X%s\r\n", (dLen+3), dLen,
    acHexBuf); // 打包 COAP 数据包 AT 命令
printf("%s\r\n", acAtBuf);
KE1_Send_AT(acAtBuf);
```


终端例程上送设备信息的字符串使用的是 JSON 格式，并使用自定义函数 `ascii2hex` 将其转换为 AT 命令支持的 ASCII 十六进制格式进行发送，如下：

`{"T":"12.36","H":"86.36","L":"520","S":"236"}` 对应的 ASCII 十六进制格式为

`7B2254223A2231322E3336222C2248223A2238362E3336222C224C223A223532302C2253223A22323336227D`

注：本例中 *T* 为温度值，*H* 为湿度值，*L* 为光照强度值，*S* 为噪音值

- 处理模组收到的下发命令信息

```
atLen = sizeof(acAtBuf);
iRet = KE1_Recv_AT(acAtBuf, &atLen, timeout);
if(4 == iRet){ //AT 命令接收到电信物联网平台下发数据
    printf("%s", acAtBuf);
    memset(acUserCmd, 0, sizeof(acUserCmd));
    cmdLen = KE1_Parse_NNMI(acAtBuf, acUserCmd);
    printf("user data[%d]:%s\r\n", cmdLen, acUserCmd);
    if(strstr(acUserCmd, "AAAA0000")){
        printf("device info upload successfully\r\n");
    }else{
        /*
         * 解析用户命令执行对应操作
         * TO-DO
         */
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7);
        HAL_Delay(1000);

        /* 向平台发送命令响应
         * AT+NMGS=5,02000A000A
         */
        acUserCmd[1] = '2';
        acUserCmd[6] = '0';
        acUserCmd[7] = '0'; acUserCmd[8] = '0';
        acUserCmd[9] = '0'; acUserCmd[10] = 0;
        printf("CmdResp:%s\r\n", acUserCmd);
        snprintf(acAtBuf, sizeof(acAtBuf), "AT+NMGS=%d,%s\r\n", 5, acUserCmd); //
        打包命令响应 AT 命令并发送
        KE1_Send_AT(acAtBuf);
    }
}
```

当使用函数 `KE1_Recv_AT` 返回值为 4 时，即为模组收到云端下发的数据信息，终端应

用程序需要解析并处理数据内容，如果数据内容为上送信息应答值则表示信息上送成功，如果为命令信息则需要解析命令并做执行，然后发送命令响应数据。

4.4.5 终端应用编译运行

请参考 *STM32 开发环境 STM32CubeIDE 安装介绍.pdf*

使用终端应用例程需要注意以下事项：

1. 终端源码会不定时更新，可关注官网更新提示。
2. 终端例程上送信息的频率为 1 小时一次，上送频率最快建议不要超过 15 秒，避免网络堵塞导致出错。
3. 终端例程没有对下发命令参数做解析，需要自行各自命令字符的解析并执行，仅仅在收到命令信息时会控制三色 LED 灯。
4. 温湿度芯片可能在未初始化成功的情况下读取信息失败，所以例程在读取到错误温湿度值时可以尝试多读几次。
5. 使用 USB 串口线连接 KE1 开发板，串口工具波特率设置为 **115200**，用于查看程序运行过程。（printf 输出信息）

4.5 KE1 微信小程序使用介绍

KE1 微信小程序联调工具。该工具是专门为 KE1 开发板功能测试的一款应用程序，同时也可以让开发者更方便的联调 KE1 开发板，加快对 5G 物联网的认知和使用。

4.5.1 准备工作

为了更好的使用 KE1 微信小程序，我们需要先完成章节 [4.4.5](#)，确保 KE1 开发板正常运行终端程序。

注：如果暂时没有完成章节 4.4，可以解压资源包中的文件 KE1BIN.zip 获取已经编译好的执行程序进行烧录运行。(该执行程序同为章节 4.4 中所介绍的例程)

4.5.2 KE1 小程序获取

使用手机微信扫描下方二维码获取 KE1 微信小程序。如图 23

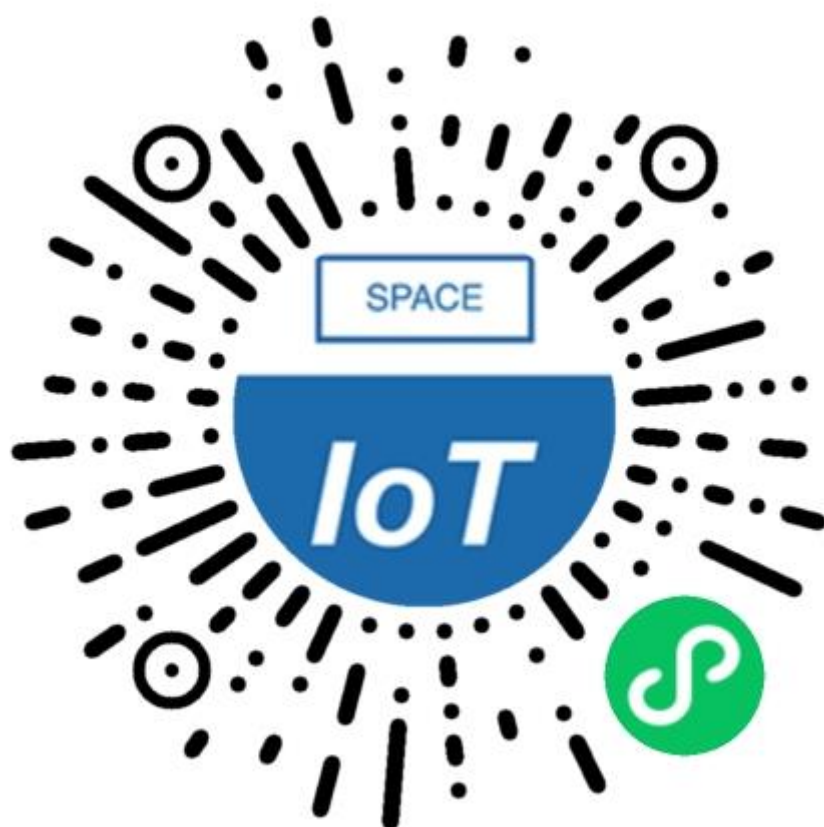


图 23 KE1 微信小程序

4.5.3 KE1 小程序使用

使用微信获取到 KE1 小程序后，在程序首页进行 KE1 设备绑定操作，具体参考页面[获取帮助](#)中的介绍，如图 24



图 24 KE1 微信小程序首页

4.5.4 KE1 小程序功能介绍

KE1 微信小程序基本功能如下：

1. KE1 终端设备绑定，解绑，昵称修改
2. KE1 终端设备上送最新信息展示（温湿度、噪音、光强、电池电压）
3. KE1 终端设备上送历史信息展示（最近 30 条）
4. 下发命令到 KE1 终端设备，控制设备 LED、2 路继电器

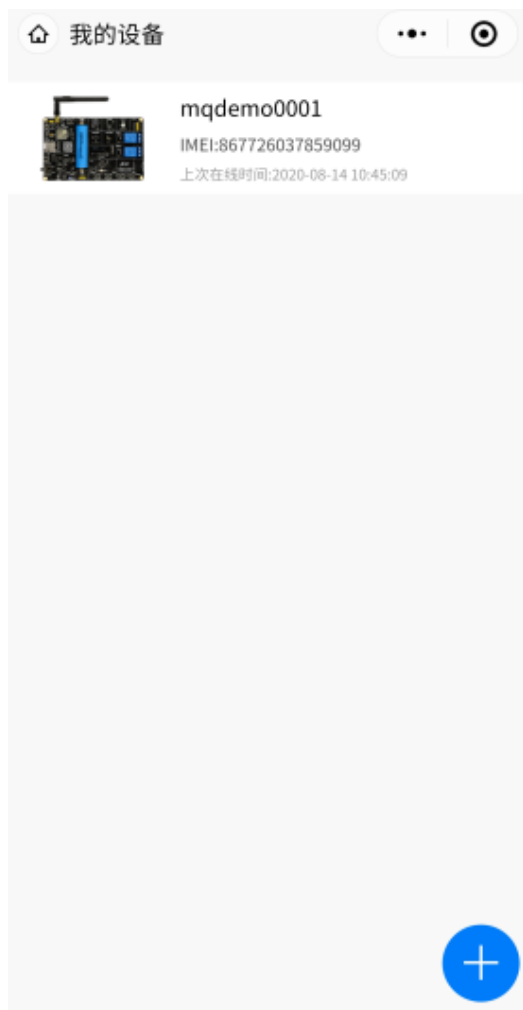


图 25 设备列表

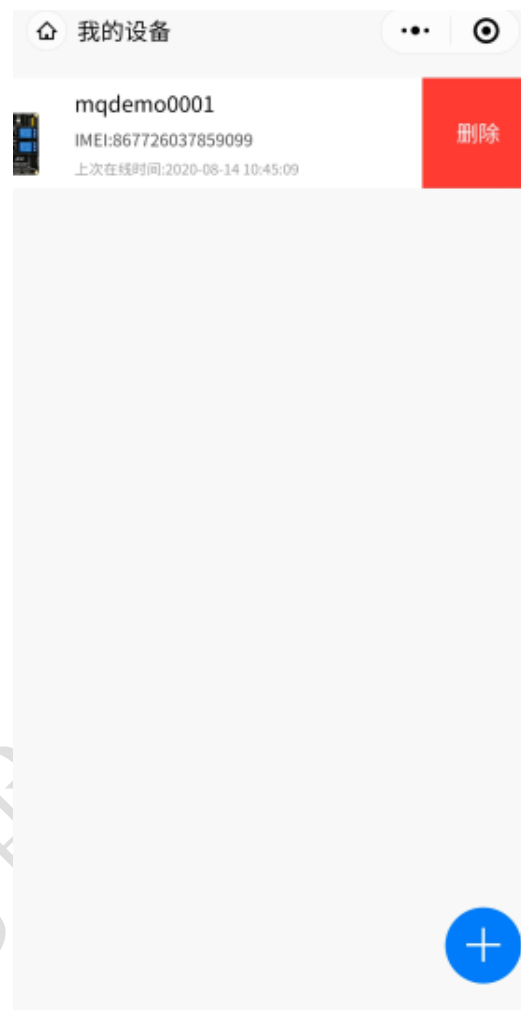


图 26 设备删除（向左滑动设备）



图 27 设备最新信息



图 28 设备控制

历史数据					
日期时间	温度	湿度	照度	噪音	电池
14/08 10:45	29.97°C	55.15%	35Lux	2020dB	3.81V
14/08 09:34	31.06°C	52.97%	20Lux	1254dB	3.74V
14/08 08:24	31.51°C	61.47%	27Lux	1157dB	3.68V
14/08 07:13	31.01°C	62.52%	28Lux	1368dB	3.76V
14/08 06:03	30.91°C	62.25%	11Lux	1355dB	3.76V
14/08 04:53	30.93°C	61.99%	11Lux	2412dB	3.87V
14/08 03:42	30.86°C	61.44%	11Lux	1430dB	3.78V
14/08 02:32	30.80°C	61.08%	11Lux	1911dB	3.83V
14/08 01:21	30.73°C	60.94%	11Lux	1955dB	3.84V
14/08 00:11	30.66°C	60.52%	11Lux	1699dB	3.84V
13/08 23:01	30.51°C	60.09%	11Lux	1851dB	3.84V
13/08 21:50	30.34°C	59.53%	11Lux	1416dB	3.81V
13/08 20:40	30.00°C	50.78%	11Lux	1068dB	3.87V

图 29 设备历史信息

5. 实验小结

本次实验中用到多种编程语言，以及不同的开发工具，通过编程实现 KE1 终端对不同传感器的数据采集并上送到电信物联网云平台。移动端 APP 或者小程序对 KE1 终端所上送的数据信息进行查看和分析，并能远程下发命令控制终端设备的各类外设（LED，继电器），从而实现了一个基本的物联网综合应用的案例。

为了更加丰富我们的实验场景，KE1 扩展包中添加更多的传感器，从而使 KE1 终端可以模拟更多的应用场景，充分的发挥大家的想象力。

完善移动端 APP 功能，并对界面进行持续性美化，使 APP 的使用更加人性化，更贴近真实场景。

6. 课后练习

● 终端实例优化

1. 完善代码流程，处理各类异常情况。
2. 增加其它信息上送云平台，并在满足指定要求时才上送信息。
3. 实现只有在外部中断事件发生时才发送信息到云平台。
4. 在使用电池供电情况下，在终端空闲时让 MCU 和模组进入低功耗模式，达到省电目的。
5. 能响应处理用户 APP 下发的各类命令/配置信息：
 - a) 控制 LED 灯
 - b) 控制继电器
 - c) 工作模式/参数修改
6. 其它功能

● 移动端 APP 优化

1. 美化 UI
2. 完善代码流程，处理各类异常情况
3. 处理并显示更多终端上送的信息
4. 完善终端控制/配置界面
 - a) 增加 LED 不同颜色显示的控制界面
 - b) 增加多路继电器的控制界面
5. 完善终端历史信息界面
6. 增加用户登录界面（模拟）
7. 增加其他界面